COMSATS University Islamabad, Attock Campus

Department of Computer Science

Course:

Information Security

Lab Assignment # 01

Submitted To

Ms. Ambreen Gul

Submitted By

Rafia Rameen

Registration No

FA24-BSE-051

Date of Submission: February 28, 2026

Task1. Write a Python program that encrypts and decrypts a message using the Caesar Cipher.

Code Explanation

- The code encrypts and decrypts a message using the Caesar Cipher.
- It shifts letters by a number you give (`shift`).
- Uppercase and lowercase letters are handled separately.
- caesar_encrypt () adds the shift to letters,
- while caesar_decrypt () subtracts the shift to get back the original message.

Output Explanation

1. Program asks for a message and a shift number.
2. It prints the Encrypted Message with letters shifted.
3. Then it prints the Decrypted Message, which is the same as the original input.

Solution

Code:

```python
#RAFIA RAMEEN (FA24-BSE-051) INFORMATION SECURITY Lab_ASSIGN_1

def caesar_encrypt(text, shift):  #FUNCTION FOR ENCRYPTION
    result = ""            #Stores encrypted message
    shift = shift % 26   # handle large shift values

    for char in text:     #Loop To Go through the message one letter at a time.
        if char.isupper():
            result += chr((ord(char) - 65 + shift) % 26 + 65)
               #Turn the letter into a number (A=0, B=1, …), add the shift,
               # make sure it stays between 0-25, then turn the number back into a letter

        elif char.islower():
            result += chr((ord(char) - 97 + shift) % 26 + 97)
            #Turn the letter into a number (a=0, b=1, …), add the shift,
            #make sure it stays between 0-25, then turn the number back into a letter

        else:
            result += char
    return result  #Return the encrypted message


def caesar_decrypt(text, shift):  #FUNCTION FOR DECRYPTION

    result = ""       #Stores decrypted message
    shift = shift % 26   # handle large shift values

    for char in text:
        if char.isupper():
            result += chr((ord(char) - 65 - shift) % 26 + 65)
            # Go back by shift positions and turn number back into capital letter

        elif char.islower():
            result += chr((ord(char) - 97 - shift) % 26 + 97)
            # Go back by shift positions and turn number back into small letter

        else:
            result += char
    return result    #Return decrypted_message

print("===== Caesar Cipher Program =====")

message = input("Enter your message: ")
shift = int(input("Enter shift value: "))     # Take shift value and convert to integer

encrypted_message = caesar_encrypt(message, shift)      #Encryption Function call
print("Encrypted Message:", encrypted_message)

decrypted_message = caesar_decrypt(encrypted_message, shift)   #Decryption Function call
print("Decrypted Message:", decrypted_message)
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\BSE Rafia\Rafia Rameen (BSE-4)\Information Security\lab work code files\LabAssignment1> & "C:\
Security/lab work code files/LabAssignment1/Question1"
===== Caesar Cipher Program =====
Enter your message: MY SELF RAFIA RAMEEN
Enter shift value: 3
Encrypted Message: PB VHOI UDILD UDPHHQ
Decrypted Message: MY SELF RAFIA RAMEEN
PS D:\BSE Rafia\Rafia Rameen (BSE-4)\Information Security\lab work code files\LabAssignment1>
```

Task2: Security Analysis

The Caesar Cipher is a very simple encryption method. It is not secure because it has only 26 possible shift values. An attacker can easily try all shifts and find the correct message. This is called a brute-force attack.

Predictable Patterns:

Common letters and words (like “the” or “and”) remain identifiable after encryption.

Frequency analysis can easily reveal the original message because the most common letters in English (e.g., E, T, A) still appear most often in the encrypted text.

It also keeps the same letter patterns, so common words can be guessed easily. Because of these weaknesses, the Caesar Cipher is only good for learning purposes and not for protecting important data. Modern encryption methods like AES (Advanced Encryption Standard) and RSA are much stronger and safer to use.